

Optimización de Layout — Carpintería

Cómo cada algoritmo crea y evoluciona un individuo

Contenido

1. Contexto del problema y distribución de la grilla
2. GA — Algoritmo Genético Tradicional
3. cGA — Algoritmo Genético Compacto
4. ES — Estrategia Evolutiva (mu + lambda)
5. Tabla comparativa de los 3 algoritmos

Contexto del Problema

La carpintería tiene una **grilla de 4×4 celdas**. Cuatro elementos son **fijos** (azul en el PDF): Mat. Prima (8), Triplay (9), Almacén (10) y Baño (11). Las **7 máquinas** (IDs 1–7) deben ubicarse en las **10 celdas disponibles** minimizando el recorrido del personal según la secuencia:

1→8, 8→1, 1→4, 4→5, 5→3, 3→4, 4→3, 3→1, 1→9, 9→1, 1→3, 3→1, 1→10

ID	Máquina	ID	Máquina
1	E. Trabajo	5	Cepillo
2	Destrozadora	6	Trompo
3	Sierra Cinta	7	Esclopeadora
4	Canteadora		

Distribución de la grilla (layout del PDF):

Col→	Col 1	Col 2	Col 3	Col 4
Fila 4	(8) Mat. Prima	Libre	Libre	Libre
Fila 3	(9) Triplay	Libre	Libre	(10) Almacén
Fila 2	(11) Baño	Libre	Libre	Libre
Fila 1	(11) Baño	Libre	Libre	Libre

Azul = elementos fijos (no se mueven). Verde = celdas disponibles para máquinas (10 celdas).

GA — Algoritmo Genético Tradicional

¿Qué es un individuo en GA?

Un **individuo** es una lista de 7 celdas, una por máquina, en el orden [máq.1, máq.2, ..., máq.7]. Representa **dónde se ubica cada máquina** en la grilla. No puede haber dos máquinas en la misma celda.

Paso 1 — Tomar las 10 celdas disponibles

El algoritmo parte de la lista de celdas que NO están bloqueadas por elementos fijos:

1	(2,1)
2	(2,2)
3	(2,3)
4	(2,4)
5	(3,1)
6	(3,2)
7	(3,3)
8	(3,4)
9	(4,1)
10	(4,2)

Paso 2 — Selección aleatoria sin repetición (random.sample)

Se eligen **7 celdas distintas** al azar de las 10 disponibles, una por máquina. Esto garantiza que no haya colisiones. La función utilizada es:

```
individuo = random.sample(celdas_disponibles, 7)
```

Paso 3 — Resultado: individuo creado

El individuo resultante asigna una celda a cada máquina. Por ejemplo:

Máquina	Nombre	Celda asignada
1	E.Trabajo	(2, 3)
2	Destrozadora	(3, 1)
3	Sierra Cinta	(3, 3)
4	Canteadora	(2, 2)
5	Cepillo	(3, 2)
6	Trompo	(4, 2)
7	Esclopeadora	(2, 4)

¿Cómo evoluciona la población?

Población inicial	Se crean 100 individuos aleatorios con el mismo proceso.
Evaluación (fitness)	Se calcula el costo total del recorrido para cada individuo. Menor costo = mejor individuo.
Selección por torneo	Se eligen grupos de 3 individuos al azar y gana el de menor costo.
Cruce OX (Order Crossover)	Se combina el material genético de dos padres preservando el orden relativo de las celdas.
Mutación por swap	Con probabilidad 15%, se intercambian dos celdas dentro del individuo.
Elitismo	El mejor individuo de la generación siempre pasa a la siguiente.

cGA — Algoritmo Genético Compacto

¿Qué es un individuo en cGA?

El cGA **no mantiene una población explícita**. En su lugar guarda una **matriz de probabilidades P** de tamaño 7×10 (7 máquinas × 10 celdas). Cada valor $P[i][j]$ representa la probabilidad de que la **máquina i** sea asignada a la **celda j**.

Paso 1 — Inicializar P con probabilidad uniforme

Al inicio, como no se sabe nada, todas las celdas son igual de probables. La probabilidad inicial es $1/10 = 0.10$ para cada combinación máquina–celda:

$$P[i][j] = 1 / 10 = 0.10 \text{ para toda máquina } i \text{ y celda } j$$

	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10
Máq.1	0.10	0.10	0.10	0.10	0.10	0.10	0.10	0.10	0.10	0.10
Máq.2	0.10	0.10	0.10	0.10	0.10	0.10	0.10	0.10	0.10	0.10
Máq.3	0.10	0.10	0.10	0.10	0.10	0.10	0.10	0.10	0.10	0.10
Máq.4	0.10	0.10	0.10	0.10	0.10	0.10	0.10	0.10	0.10	0.10
Máq.5	0.10	0.10	0.10	0.10	0.10	0.10	0.10	0.10	0.10	0.10
Máq.6	0.10	0.10	0.10	0.10	0.10	0.10	0.10	0.10	0.10	0.10
Máq.7	0.10	0.10	0.10	0.10	0.10	0.10	0.10	0.10	0.10	0.10

Paso 2 — Muestrear un individuo desde P

Para crear un individuo se procesa **cada máquina en orden**. Para la máquina i, se elige una celda disponible según las probabilidades de la fila $P[i]$, sin repetir celdas ya usadas:

Máquina 1 (E. Trabajo)	Todas las celdas disponibles con prob. 0.10. Se sortea → resulta (2,3).
Máquina 2 (Destrozadora)	Se excluye (2,3). Quedan 9 celdas. Se sortea → resulta (3,1).
Máquina 3 (Sierra Cinta)	Se excluyen (2,3) y (3,1). Quedan 8 celdas. Se sortea → resulta (3,3).
... y así	Se continúa hasta asignar todas las 7 máquinas sin repetir celda.

Paso 3 — Competencia y actualización de P

Se generan **2 individuos** con el proceso anterior y se evalúa cuál tiene menor costo de recorrido. El ganador **incrementa** sus probabilidades en P y el perdedor las **reduce** en $1/N$ ($N = 200$, tamaño virtual):

Si ganador usó celda j para máquina i: $P[i][j] += 1/200 = +0.005$ (se vuelve más probable)
 $P[i][j] -= 1/200 = -0.005$ (para el perdedor: menos probable)

Al converger, cada fila de P tendrá un valor cercano a 1.0 en una sola celda y 0.0 en el resto: el algoritmo ha encontrado la mejor asignación para esa máquina.

ES — Estrategia Evolutiva (mu + lambda)

¿Qué es un individuo en ES?

Un individuo en ES es un **vector de 7 números reales (flotantes)**, uno por máquina. Cada número representa un **índice continuo** dentro de la lista de celdas disponibles. Al momento de evaluar, se convierte al espacio discreto (una celda real).

Paso 1 — Crear individuo en espacio real

Se generan 7 índices enteros distintos (sin repetición) del rango [0, 9] (porque hay 10 celdas disponibles) y se convierten a flotantes:

```
indices = random.sample(range(10), 7) → [2, 7, 1, 5, 8, 0, 4] individuo = [float(i) for i in indices] → [2.0, 7.0, 1.0, 5.0, 8.0, 0.0, 4.0]
```

Paso 2 — Decodificar individuo a asignación discreta

Para evaluar el costo, el vector real se convierte a celdas reales. El proceso ordena las máquinas por su valor flotante y asigna celdas en ese orden, tomando la celda del índice más cercano no usada:

Máquina	Valor flotante	Índice redondeado	Celda asignada
Máq.6	0.0	0	(2,1)
Máq.3	1.0	1	(2,2)
Máq.1	2.0	2	(2,3)
Máq.7	4.0	4	(3,1)
Máq.4	5.0	5	(3,2)
Máq.2	7.0	7	(3,4)
Máq.5	8.0	8	(4,1)

Paso 3 — Mutación Gaussiana (cómo evolucionan los individuos)

Los hijos se generan añadiendo **ruido gaussiano $N(0, \sigma)$** a cada valor del vector padre. Esto desplaza los índices levemente, explorando celdas vecinas:

```
hijo[i] = padre[i] + random.gauss(0, sigma) Ejemplo con sigma=1.0: padre = [2.0, 7.0, 1.0, 5.0, 8.0, 0.0, 4.0] ruido = [+0.3, -1.2, +0.8, +0.1, -0.5, +1.1, -0.2] hijo = [2.3, 5.8, 1.8, 5.1, 7.5, 1.1, 3.8]
```

Paso 4 — Selección (mu + lambda)

Con **mu=10 padres** se generan **lambda=50 hijos**. Luego se evalúan todos (60 individuos) y los **10 mejores** se convierten en los nuevos padres. Esto garantiza que la calidad nunca empeora entre generaciones.

Paso 5 — Adaptación del paso sigma (Regla del 1/5)

Si más del 20% de los hijos son mejores que sus padres, sigma **aumenta** (exploración más amplia). Si menos del 20% mejoran, sigma **disminuye** (refinamiento fino). Esto adapta automáticamente la intensidad de la búsqueda:

```
tasa_exito > 0.20 → sigma /= 0.85 (sigma crece) tasa_exito < 0.20 → sigma *= 0.85 (sigma decrece)
```

Comparativa de los 3 Algoritmos

Aspecto	GA	cGA	ES
Representación	Lista de 7 celdas (discretas)	Matriz P 7×10 de probabilidades	Vector de 7 flotantes (continua)
Creación de individuo	random.sample(celdas, 7)	Sorteo ponderado por P	random.sample(range(10),7) → float
Tamaño de población	100 individuos explícitos	20 individuos por iteración (virtual N=200)	10 padres + 50 hijos = 60
Operador principal	Cruce OX + Mutación swap	Actualización de probabilidades	Mutación Gaussiana N(0,σ)
Memoria	Alta (guarda 100 individuos)	Muy baja (solo matriz 7×10)	Media (guarda 60 individuos)
Adaptación	No adapta parámetros	P converge sola	Regla del 1/5 adapta sigma
Mejor costo (Manhattan)	18.00	18.00	18.00
Mejor costo (Euclidiana)	15.66	15.66	15.66

Conclusión

Los tres algoritmos convergen al **mismo óptimo** (costo 18.00 Manhattan / 15.66 Euclidiana) gracias a que el espacio de búsqueda es relativamente pequeño (10 celdas para 7 máquinas = $10!/(10-7)! = 604,800$ permutaciones posibles). La diferencia está en **cómo exploran** ese espacio: GA con diversidad genética, cGA con aprendizaje probabilístico, y ES con perturbaciones adaptativas.